PaperMate（论文智能管理与知识库系统）软件架构文档

PaperMate 软件架构文档

日期	版本	说明	作者
2026 年 7 月 17 日	0.0	初次填写	软工原第五小组
2026 年 7 月 21 日	1.0	第一次修改	软工原第五小组

---

1. 简介

1.1 目的

本文档从软件架构角度对 PaperMate（论文智能管理与知识库系统）进行综合说明，记录系统在功能分解、模块协作、数据存储、部署方式、技术选型和质量属性方面的重要设计决策。

本文档主要面向以下读者：

- **项目组开发人员**：据此理解模块边界、接口关系和代码组织方式；
- **测试人员**：据此设计功能、接口、性能、可靠性和安全性测试；
- **项目评审人员与教师**：据此评估系统架构是否能够支撑需求；
- **系统部署与维护人员**：据此完成环境配置、部署、日志检查和数据备份。

本文档按照用例视图、逻辑视图、部署视图、进程视图、实现视图、技术视图、数据视图、算法视图以及质量属性视图组织内容。

1.2 系统范围

PaperMate 是一套面向科研人员和学生的 Web 论文管理与知识服务系统，核心能力包括：

1. 定时从 arXiv 按学科、关键词或时间范围抓取论文；
2. 对论文元数据进行分类、入库和去重；
3. 解析论文 PDF 或 HTML，生成摘要、核心概念、研究方法和实验结论等结构化内容；
4. 建立论文、作者、主题、概念和引用关系之间的关联；
5. 提供关键词检索、条件筛选和语义检索；
6. 基于论文知识库进行自然语言问答，并返回论文出处或内容定位；
7. 支持收藏、笔记、阅读历史和订阅等学习管理功能；
8. 支持系统管理员查看任务状态、日志和异常信息。

1.3 架构目标

系统架构应满足以下目标：

- 保证论文抓取、解析、检索和问答功能之间的清晰分层；
- 避免各模块直接耦合，便于五人小组并行开发；
- 在有限开发周期内优先完成核心 MVP；
- 支持后续增加新的论文来源、解析算法、嵌入模型和大语言模型；
- 对外部服务故障、任务失败和重复抓取具备基本恢复能力；
- 在 100 并发场景下，使常规检索和学习管理接口在 3 秒内返回，不包含大语言模型生成时间。

1.4 架构原则

1. **采用模块化单体架构。** 当前项目规模和团队人数有限，不采用微服务，以降低部署、接口治理和分布式事务复杂度。
2. **前后端分离。** 前端通过 RESTful API 调用后端，前端不直接访问数据库或外部模型服务。
3. **异步处理耗时任务。** 论文抓取、PDF 下载、文本解析、知识抽取和向量生成由后台任务执行，避免阻塞普通请求。
4. **数据库统一管理核心数据。** PostgreSQL 保存用户、论文、任务、学习记录和关系数据，pgvector 保存文本向量。
5. **外部能力通过适配器接入。** arXiv、LLM、嵌入模型和 PDF 解析器均通过独立适配层访问，便于替换实现。
6. **回答必须绑定证据。** 智能问答优先使用已入库论文内容，并返回论文、章节或文本片段来源。

7. **优先实现可维护性。** 统一目录结构、接口规范、异常处理、日志格式和配置管理方式。

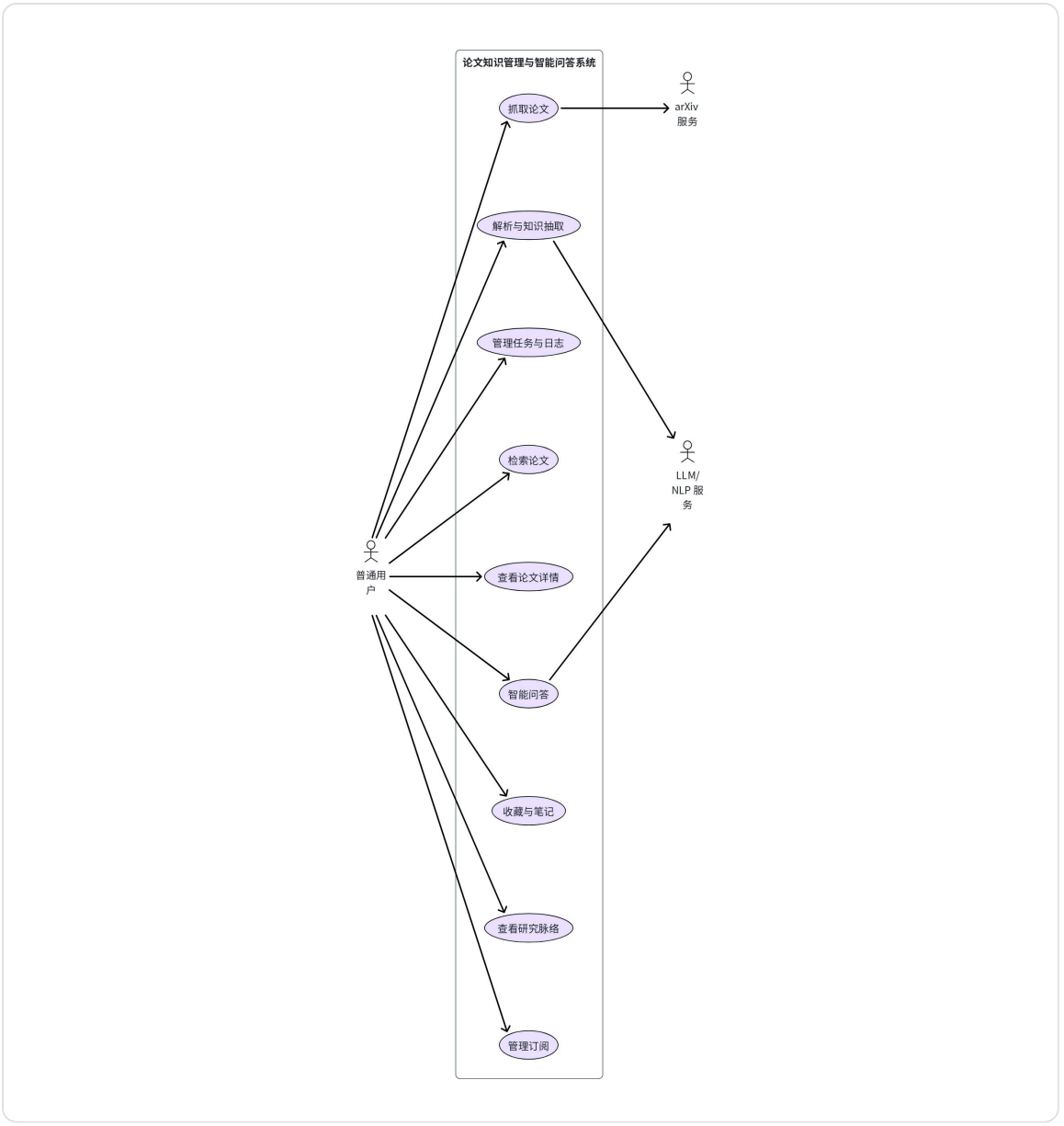
---

2. 用例视图

2.1 参与者

参与者	职责
普通用户	检索论文、查看论文详情、进行智能问答、收藏论文、撰写笔记、查看阅读历史和管理订阅。
arXiv 服务	提供论文元数据、论文链接和 PDF 文件。
LLM/NLP 服务	提供文本摘要、知识抽取、嵌入生成和自然语言回答能力。
定时调度器	按计划触发论文抓取、解析、索引更新和订阅推送任务。

2.2 核心用例



2.3 架构上重要的用例

2.3.1 论文自动抓取与入库

该用例贯穿任务调度、arXiv 适配器、论文服务、去重服务和数据库，是系统数据来源的基础。

基本流程：

1. 定时调度器创建抓取任务；
2. arXiv 适配器根据关键词、类别、时间和分页参数请求论文元数据；
3. 抓取服务对返回结果进行规范化；
4. 去重服务根据 arXiv ID、版本号和标题等信息判断论文是否已存在；
5. 新论文写入数据库，已有论文更新版本和元数据；

6. 系统为新论文创建 PDF 下载与解析任务；
7. 系统记录任务结果、失败原因和重试次数。

2.3.2 论文检索与详情查看

该用例涉及前端搜索页面、检索 API、论文服务、PostgreSQL、pgvector 和权限控制。

基本流程：

1. 用户输入关键词或选择作者、类别、时间、标签等筛选条件；
2. 后端校验参数并执行结构化检索；
3. 用户选择语义检索时，系统生成查询向量并执行向量相似度搜索；
4. 系统合并、排序并分页返回结果；
5. 用户进入详情页后，系统返回论文元数据、结构化内容、概念、方法、实验结论和关系信息。

2.3.3 自然语言论文问答

该用例涉及问答控制器、检索服务、向量检索、知识库、LLM 适配器、出处校验和问答历史，是架构范围最广的核心用例。

前置条件：

- 系统中已有论文元数据、结构化解析内容和向量索引；
- LLM/NLP 服务可用；
- 用户已进入智能问答页面。

基本流程：

1. 用户输入自然语言问题并提交；
2. 系统检查问题是否为空、长度是否合理；
3. 系统识别关键词、研究主题和可能涉及的论文范围；
4. 检索服务从论文元数据、结构化内容和向量索引中召回相关内容；
5. 重排序模块筛选最相关的论文片段；
6. 系统将问题、候选片段和回答约束发送给 LLM/NLP 服务；
7. LLM/NLP 服务生成基于论文内容的回答；

8. 出处校验模块检查回答中的主要结论是否能够绑定论文来源；
9. 系统返回答案、论文标题、作者、发布时间、arXiv 链接及片段位置；
10. 系统保存本次问答记录。

备选与异常流程：

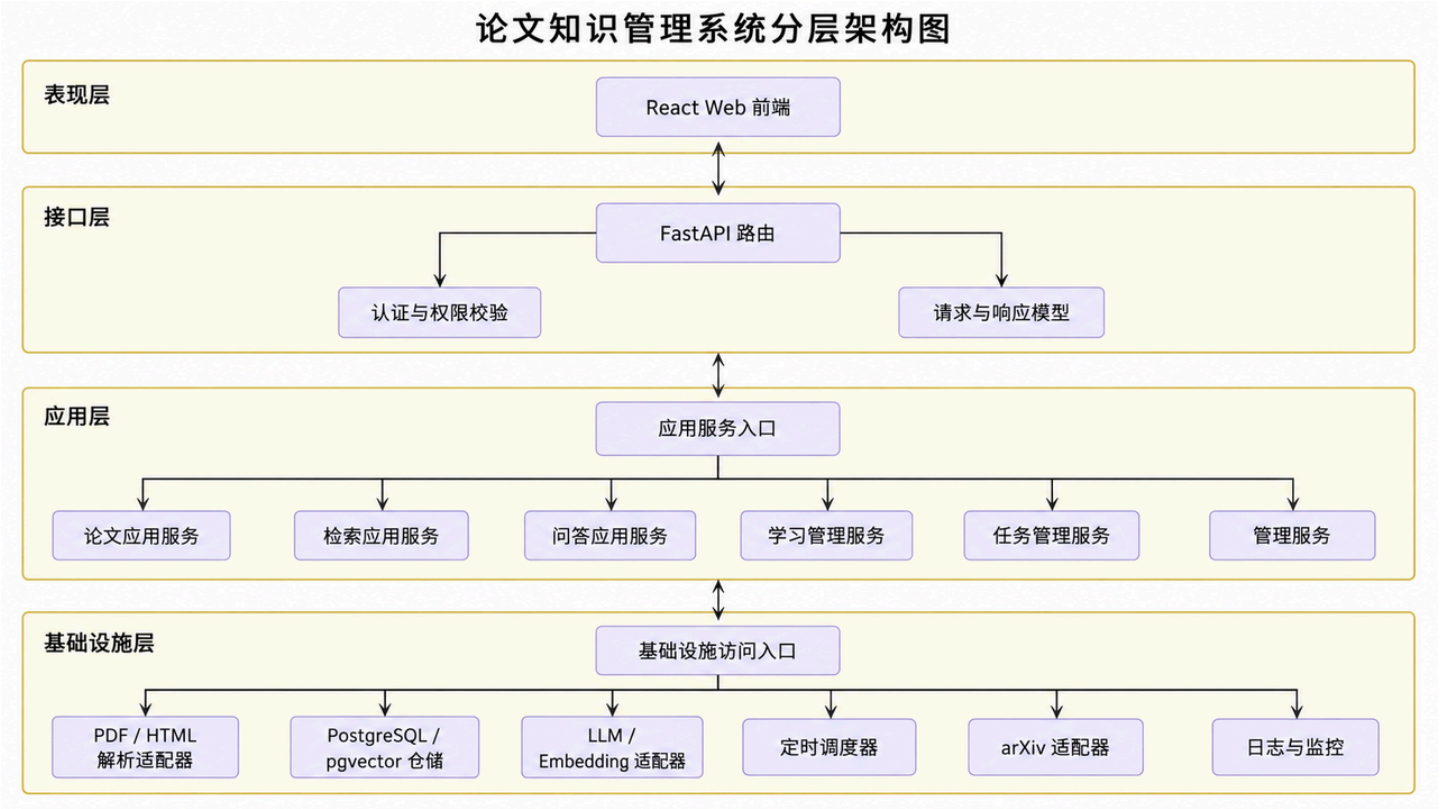
- 输入为空：返回参数错误，不调用检索和 LLM；
- 未检索到可靠内容：提示“暂未找到相关论文依据”，不强行生成确定性答案；
- LLM 调用失败：返回已检索到的论文和片段，并提示稍后重试；
- 回答缺少可靠出处：降低回答置信度，优先展示可验证片段；
- 请求超时：终止外部调用，记录日志并返回超时提示。

---

3. 逻辑视图

3.1 总体逻辑架构

系统采用“表现层—接口层—应用层—领域层—基础设施层”的分层结构，并在单一后端应用中按照业务领域划分模块。



3.2 子系统划分

以下内容可直接替换原来的 3.2.1—3.2.7，将系统按照架构图划分为四个层级子系统。

3.2.1 表现层子系统

职责：

- 为普通用户提供统一的 Web 操作界面；
- 提供用户注册、登录、退出和个人设置界面；
- 展示论文列表、论文详情、研究脉络和论文 Wiki；
- 提供论文检索、智能问答、收藏、笔记和阅读进度等交互界面；
- 提供订阅管理、任务管理、日志查看和系统管理界面；
- 负责前端路由、页面状态管理、表单校验和交互反馈；
- 调用接口层提供的 API，并将响应结果转换为可视化页面内容。

重要类或组件：

- `React Web 前端`：表现层的统一入口；
- `Router`：前端页面路由组件；
- `LoginPage`：用户登录页面；
- `PaperListPage`：论文列表与检索结果页面；
- `PaperDetailPage`：论文详情和结构化 Wiki 页面；
- `QuestionAnswerPage`：智能问答页面；
- `LearningManagementPage`：收藏、笔记、订阅和阅读进度管理页面；
- `TaskManagementPage`：任务状态和系统日志管理页面；
- `ApiClient`：封装前端对后端接口的调用；
- `StateStore`：管理用户状态、页面状态和共享数据。

3.2.2 接口层子系统

职责：

- 对外提供基于 FastAPI 的 HTTP API；
- 接收表现层发送的请求，并将请求分发给对应的应用服务；
- 对请求参数、路径参数和查询参数进行校验；
- 完成用户身份认证、访问令牌校验和权限控制；
- 将请求数据转换为应用层可处理的对象；

- 将应用层返回的数据转换为统一的响应格式；
- 对业务异常、参数异常和系统异常进行统一处理；
- 生成和维护 OpenAPI 接口文档。

重要类或组件：

- `FastAPI 路由`：接口层统一入口，负责请求接收和路由分发；
- `AuthRouter`：处理登录、退出和身份认证相关接口；
- `PaperRouter`：处理论文查询、详情和管理接口；
- `SearchRouter`：处理关键词检索和语义检索接口；
- `QuestionAnswerRouter`：处理智能问答接口；
- `LearningRouter`：处理收藏、笔记、订阅和阅读记录接口；
- `TaskRouter`：处理任务创建、查询和管理接口；
- `认证与权限校验`：执行令牌验证和角色权限检查；
- `RequestModel`：定义请求参数模型；
- `ResponseModel`：定义统一响应数据模型；
- `ExceptionHandler`：将系统异常转换为标准 HTTP 响应。

3.2.3 应用层子系统

职责：

- 作为系统业务用例的统一组织和编排层；
- 接收接口层传入的请求，调用相应的应用服务完成业务处理；
- 协调论文、用户、任务、检索、问答和学习管理等功能；
- 创建和执行论文抓取、下载、解析、知识抽取和索引任务；
- 完成论文元数据管理、去重、分类、标签和状态更新；
- 组织关键词检索、向量检索和混合排序流程；
- 组织问题分析、证据检索、提示词构造、模型调用和出处绑定流程；
- 管理收藏、笔记、阅读记录、主题订阅和简单推荐；
- 管理用户、角色、系统任务和运行状态；
- 控制应用级事务，协调多个基础设施组件共同完成业务操作。

重要类或组件：

- `应用服务入口`：应用层统一访问入口，负责将请求分发给具体应用服务；

- `PaperApplicationService`：论文应用服务，负责论文抓取、管理、去重、解析和知识组织；
- `SearchApplicationService`：检索应用服务，负责关键词检索、向量检索和混合排序；
- `QuestionAnswerApplicationService`：问答应用服务，负责证据检索、上下文构造、模型调用和回答生成；
- `LearningManagementService`：学习管理服务，负责收藏、笔记、阅读进度和订阅管理；
- `TaskManagementService`：任务管理服务，负责任务创建、状态流转、失败重试和结果查询；
- `ManagementService`：管理服务，负责用户、角色、权限、日志和系统状态管理；
- `PaperDeduplicationService`：论文去重服务；
- `HybridRanker`：关键词结果和向量结果的融合排序组件；
- `PromptBuilder`：智能问答提示词和上下文构造组件；
- `CitationVerifier`：回答出处和引用信息校验组件。

3.2.4 基础设施层子系统

职责：

- 为应用层提供数据库、外部服务、文件解析、模型调用和任务调度能力；
- 使用 PostgreSQL 持久化用户、论文、任务、笔记和关系数据；
- 使用 pgvector 保存文本向量并支持向量相似度检索；
- 调用 arXiv API 获取论文元数据和下载地址；
- 下载、校验并解析论文 PDF 或 HTML 文档；
- 调用 LLM 完成摘要生成、知识抽取和智能问答；
- 调用 Embedding 模型生成文本向量；
- 执行定时抓取、解析、索引和重试任务；
- 记录系统运行日志、异常日志和任务执行信息；
- 提供系统健康检查、运行监控和故障诊断能力；
- 隔离数据库、模型服务和外部 API 的具体实现，避免应用层直接依赖第三方技术细节。

重要类或组件：

- `基础设施访问入口`：基础设施能力的统一访问入口；
- `PostgreSQLRepository`：关系数据持久化仓储；
- `PgvectorRepository`：文本向量存储与相似度检索仓储；

- `ArxivAdapter`：arXiv 外部服务适配器；
- `PdfParserAdapter`：PDF 文档解析适配器；
- `HtmlParserAdapter`：HTML 文档解析适配器；
- `LLMAdapter`：大语言模型调用适配器；
- `EmbeddingAdapter`：文本向量模型调用适配器；
- `TaskScheduler`：定时任务调度器；
- `TaskExecutor`：后台任务执行组件；
- `RetryPolicy`：任务失败重试策略；
- `LoggingService`：结构化日志记录组件；
- `MonitoringService`：系统运行状态监控组件；
- `HealthCheckService`：基础健康检查服务。

3.3 模块依赖规则

1. 表现层只能调用接口层，不得直接访问数据库；
2. 接口层负责参数校验、认证和响应转换，不实现复杂业务；
3. 应用层负责用例编排，可以调用领域对象和基础设施接口；
4. 基础设施层实现仓储和外部服务接口；
5. 论文、问答、学习管理等模块之间通过应用服务或明确接口协作，不直接修改其他模块的数据表；
6. 外部服务失败时，由适配器统一转换为系统内部异常。

---

4. 部署视图

4.1 当前部署配置

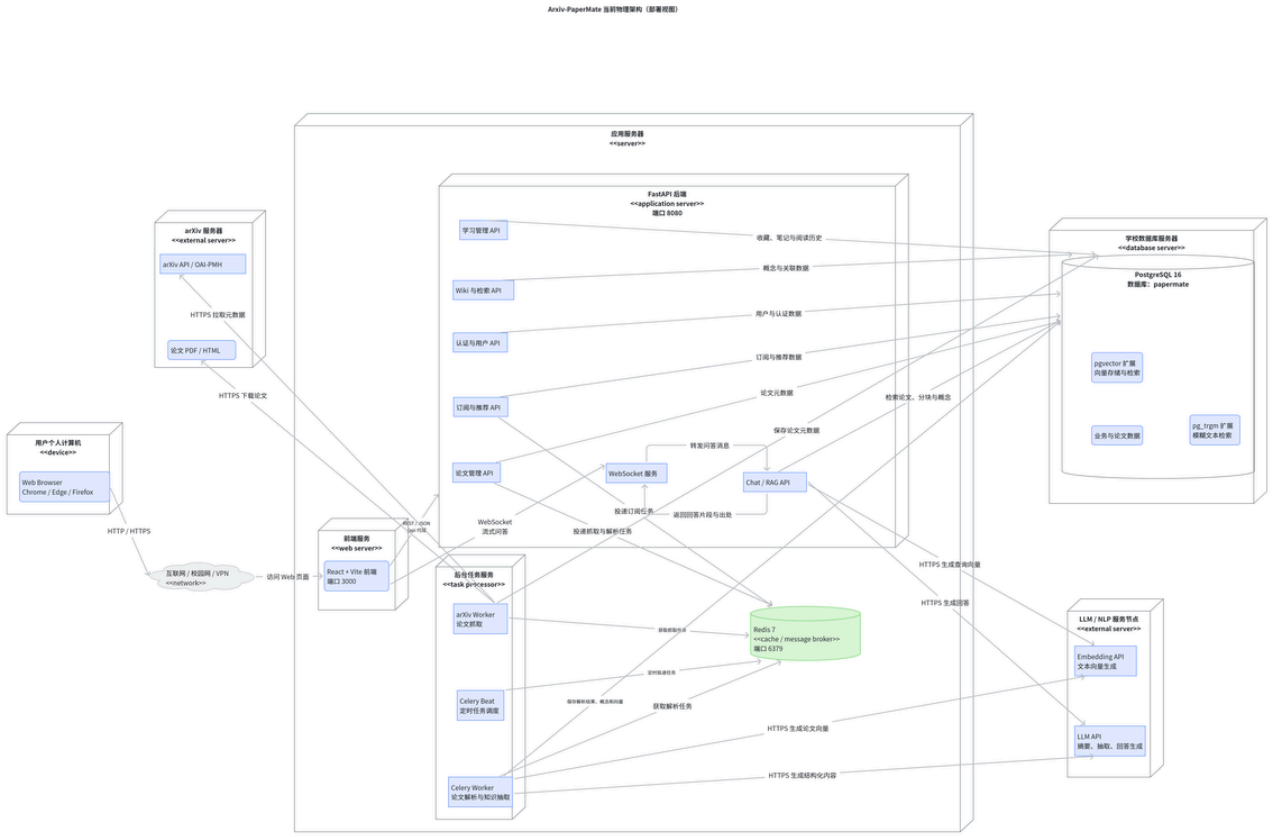
Arxiv-PaperMate 当前采用 B/S 架构。用户通过个人计算机上的 Web 浏览器访问系统，无需在用户终端安装专用客户端。

系统的 React/Vite 前端、FastAPI 后端、Celery 后台任务服务和 Redis 部署在应用服务器上。其中，前端服务默认使用 3000 端口提供 Web 页面，并将 `/api` 请求代理到 FastAPI；FastAPI 默认使用 8080 端口提供认证、论文管理、Wiki、学习管理、订阅推荐和智能问答等接口。

后台任务服务由 Celery Worker、arXiv Worker 和 Celery Beat 构成。Celery Worker 负责论文解析与知识抽取，arXiv Worker 负责论文抓取，Celery Beat 负责定时触发抓取和订阅等任务。Redis作为消息代理、任务队列和缓存，协调 FastAPI 与各后台任务执行器。

PostgreSQL 16 独立部署在学校数据库服务器上，数据库名称为 `papermate`。数据库启用了 `pgvector` 和 `pg_trgm` 扩展，分别支持向量存储与相似度检索，以及论文标题、作者和摘要等文本的模糊检索。

系统通过互联网访问相互独立的 arXiv 服务器和 LLM/NLP 服务节点。arXiv 服务器提供论文元数据、PDF 和 HTML 内容；LLM/NLP 服务提供摘要生成、知识抽取、回答生成和文本向量化能力。



4.2 节点与软件映射

物理节点	部署的软件或构件	主要职责
用户个人计算机	Chrome、Edge、Firefox 等 Web 浏览器	展示系统界面、接收用户输入、发送 API 请式问答连接
应用服务器	React、Vite 前端服务	提供 Web 页面和前端静态资源，并将 /api FastAPI
应用服务器	FastAPI 后端服务	提供认证、用户、论文、Wiki、学习管理、智能问答 API
应用服务器	WebSocket 服务	建立智能问答的双向通信连接，向前端持续段和论文出处
应用服务器	Celery Worker	执行论文 PDF 解析、结构化内容生成、概念化任务
应用服务器	arXiv Worker	调用 arXiv 服务，抓取论文元数据并下载论
应用服务器	Celery Beat	按设定时间触发论文抓取、解析和订阅推送
应用服务器	Redis 7	提供消息代理、异步任务队列、任务状态存力
学校数据库服务器	PostgreSQL 16	保存用户、论文、概念、引用、笔记、收藏和订阅等持久化数据
学校数据库服务器	pgvector 扩展	保存论文分块向量和查询向量，支持向量相
学校数据库服务器	pg_trgm 扩展	支持标题、作者和摘要等字段的模糊文本检
arXiv 服务器	arXiv API、OAI-PMH、论文 PDF/HTML	提供论文元数据、摘要、分类、原始论文和
LLM/NLP 服务节点	LLM API	提供论文摘要、概念与方法抽取、对比分析生成
LLM/NLP 服务节点	Embedding API	将论文分块和用户查询转换为文本向量

4.3 网络与通信

浏览器与前端服务

用户浏览器通过互联网、校园网或 VPN 访问应用服务器上的前端服务。当前开发或课程演示环境可以使用 HTTP，正式公开部署时应使用 HTTPS。

前端与FastAPI后端

前端通过 RESTful API 调用 FastAPI，主要使用 JSON 交换数据。Vite 将浏览器发出的 `/api` 请求代理到默认运行在 8080 端口的 FastAPI 服务。

前端与WebSocket服务

智能问答需要持续返回回答内容时，前端通过 WebSocket 与后端建立双向连接。WebSocket 服务接收问题并转交 Chat/RAG 服务，然后将回答片段和论文出处逐步返回给浏览器。

后端与数据库服务器

FastAPI、Celery Worker 和 arXiv Worker通过 PostgreSQL 数据库协议访问学校数据库服务器。FastAPI 负责读取和修改业务数据，后台任务执行器负责保存抓取结果、解析结果、概念关系和论文向量。

数据库不直接向用户浏览器开放，只允许应用服务器访问。数据库连接应通过校园网、VPN、SSH 隧道或 TLS 进行保护。

后端与Redis

FastAPI 将抓取、解析和订阅等耗时任务投递到 Redis。Celery Worker 和 arXiv Worker 从 Redis 获取任务，Celery Beat 则按照预定时间向 Redis 投递定时任务。

后台任务与arXiv

arXiv Worker通过 HTTPS 调用 arXiv API 或 OAI-PMH 接口，获取论文元数据，并根据论文链接下载 PDF 或 HTML 内容。

后端与LLM/NLP服务

Celery Worker通过 HTTPS 调用 LLM API，生成论文摘要并抽取概念、方法和实验结论；通过 Embedding API 生成论文分块向量。Chat/RAG 服务调用 Embedding API 生成查询向量，并调用 LLM API 生成带有论文出处的回答。

敏感信息管理

数据库账号、JWT 密钥、LLM API 密钥和管理员密钥只能保存在后端环境变量或服务器配置中，不得写入前端代码，也不得发送到用户浏览器。

4.4 部署限制与改进方向

当前部署结构适用于课程开发、系统演示和中小规模使用，但仍存在以下限制：

Vite不适合作为正式生产Web服务器

当前由 Vite 提供前端页面和 API 代理，适合开发环境，但不适合直接承担生产环境流量。后续可将 React 构建产物部署到 Nginx、对象存储或 CDN，并由 Nginx 反向代理 FastAPI 和 WebSocket。

应用服务器存在单点故障

前端、FastAPI、Redis 和 Celery 当前集中部署在同一台应用服务器上。应用服务器出现故障时，页面访问、API 和后台任务都会受到影响。后续可以将这些服务拆分到不同服务器或容器中。

Redis与业务服务共享服务器

Redis承担缓存和任务消息代理职责。如果后台任务数量增加，Redis可能与 FastAPI 和 Celery 竞争内存及 CPU 资源。后续可将 Redis 独立部署，并配置持久化、访问密码和故障恢复机制。

后台任务处理能力有限

当前 Celery Worker 数量和并发能力有限，大量论文同时抓取和解析时可能出现任务堆积。后续可增加 Worker 实例，并将抓取、解析、向量化和订阅推送划分到不同任务队列。

数据库依赖校园网络

PostgreSQL 部署在学校服务器上，应用服务器必须通过校园网、VPN 或其他受控网络访问。网络中断会导致论文查询、用户登录和任务处理失败。后续应增加数据库连接池、断线重连、定期备份和监控

告警。

外部服务具有不确定性

arXiv 和 LLM/NLP 均属于外部服务，可能出现访问限流、网络超时或接口不可用。后续应增加请求重试、超时控制、限流、熔断和降级策略，并缓存已经获取的论文和模型处理结果。

安全能力需要进一步完善

正式部署时应统一启用 HTTPS，限制数据库和 Redis 的访问来源，定期轮换密钥，并增加接口限流、日志审计、数据库备份和敏感数据保护机制。

后续扩展方向

当用户量和论文数量增加后，可以采用容器化部署，将前端、FastAPI、Redis 和各类 Worker分别部署；同时增加负载均衡、多实例 FastAPI、独立任务节点、数据库备份和监控系统，从而提升系统的性能、可靠性和可维护性。

---

5. 进程视图

5.1 运行单元

系统运行时包含以下主要进程或执行单元：

1. **前端开发服务器或静态资源服务进程**：提供 React 页面；
2. **FastAPI Web 进程**：处理认证、检索、详情、问答和学习管理请求；
3. **后台任务执行进程**：执行抓取、下载、解析、抽取和向量化任务；
4. **定时调度线程**：定期创建抓取和索引任务；
5. **PostgreSQL 数据库进程**：处理事务、索引和向量检索；
6. **外部 LLM/NLP 服务进程**：由第三方或本地模型服务提供。

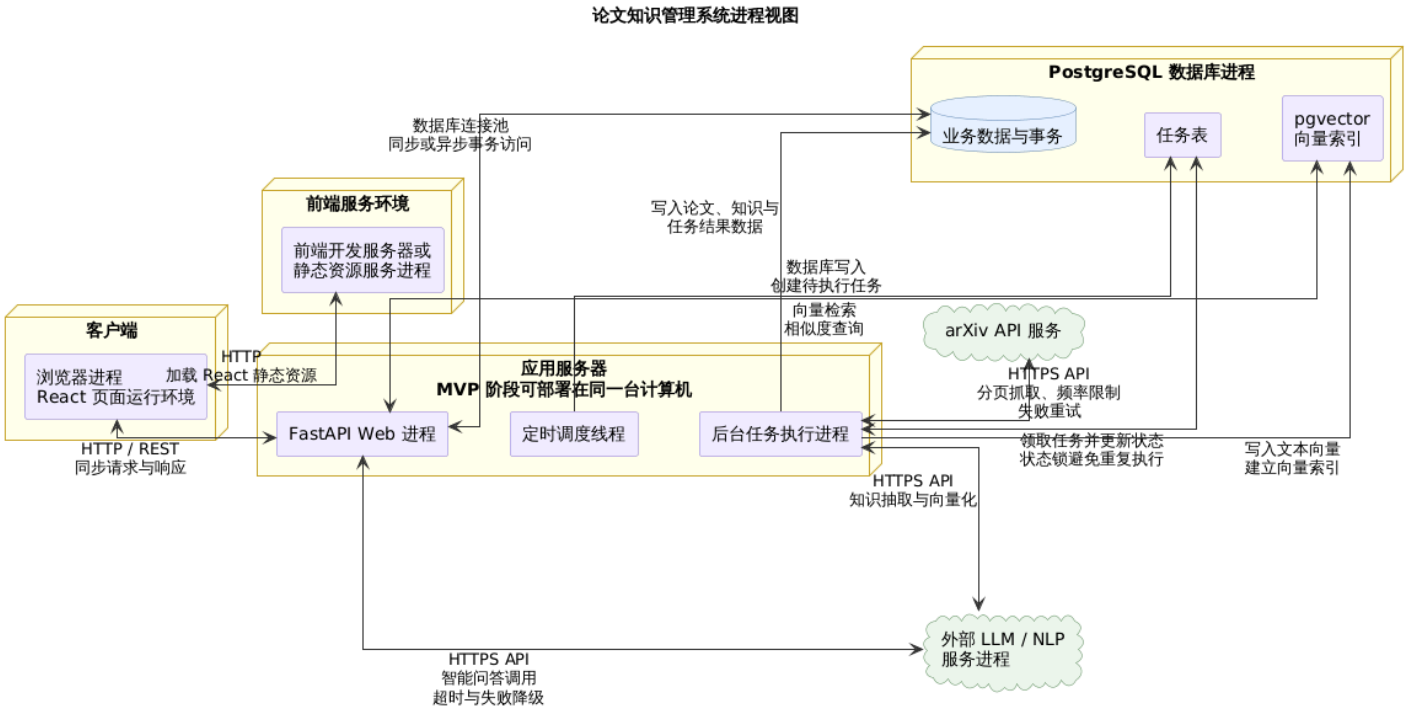
在 MVP 阶段，FastAPI、调度器和后台任务执行器可以部署在同一台计算机，但在逻辑上保持独立，避免耗时任务占用 Web 请求线程。

5.2 通信模式

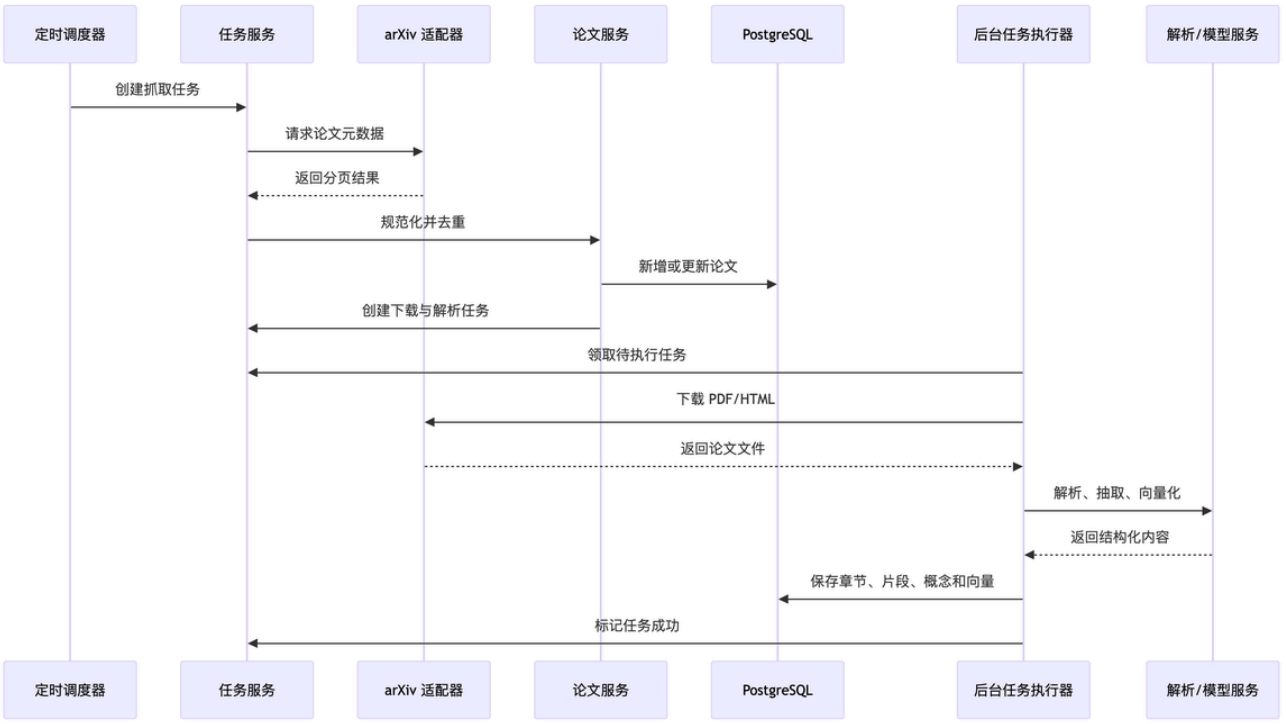
通信双方	通信方式	特征

浏览器—FastAPI	HTTP/REST	同步请求—响应。
FastAPI—PostgreSQL	数据库连接池	同步或异步数据库访问，使用事务。
调度器—任务表	数据库读写	创建待执行任务。
任务执行器—任务表	状态轮询或领取任务	采用任务状态和锁避免重复执行。
后端—LLM	HTTPS API	外部同步调用，设置超时和失败降级。
任务执行器—arXiv	HTTPS API	分页抓取，限制频率，失败重试。

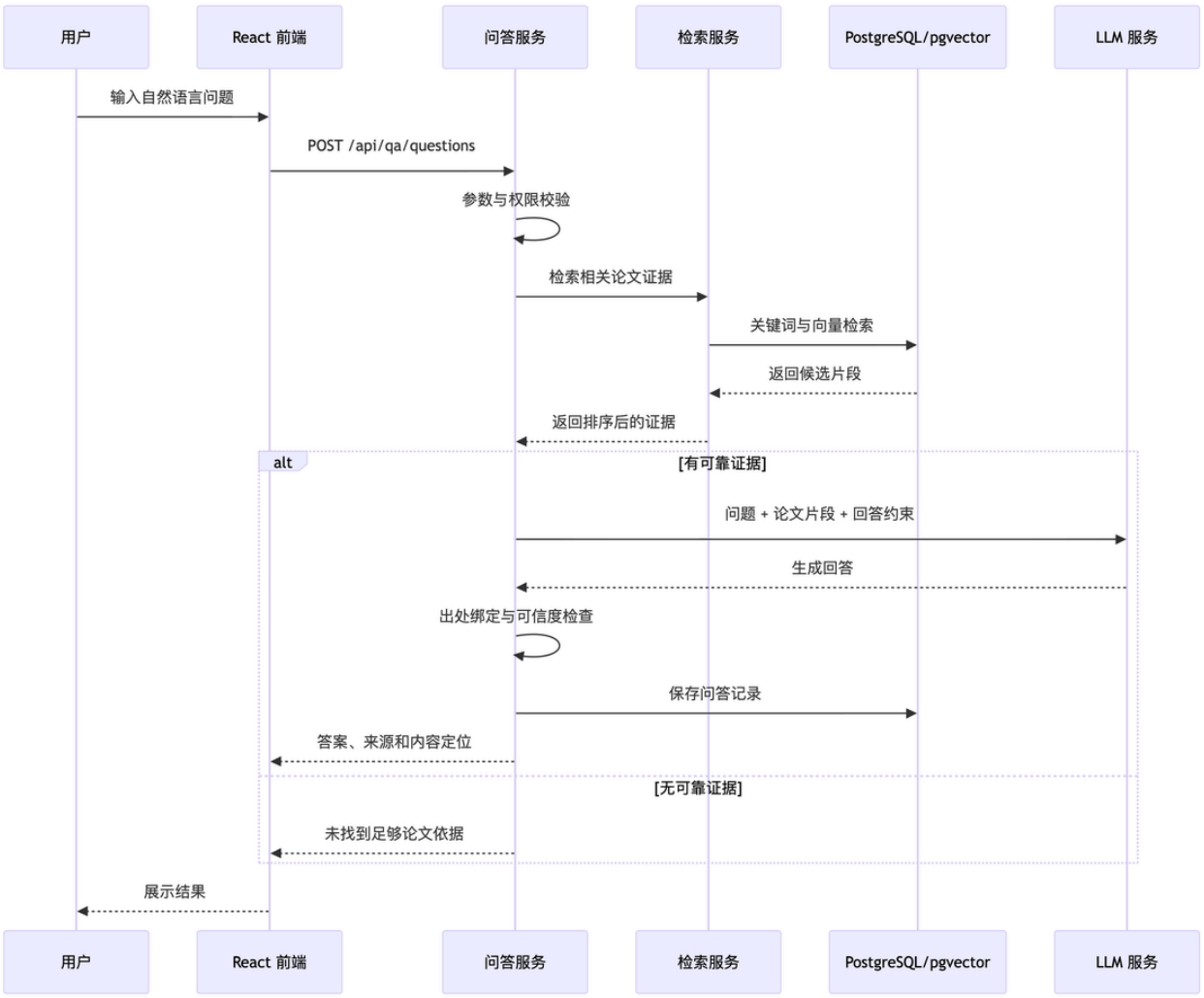
5.3 进程类图



5.4 论文处理过程


定时调度器
任务服务
arXiv 适配器
论文服务
PostgreSQL
后台任务执行器
解析/模型服务
创建抓取任务
请求论文元数据
返回分页结果
规范化并去重
新增或更新论文
创建下载与解析任务
领取待执行任务
下载 PDF/HTML
返回论文文件
解析、抽取、向量化
返回结构化内容
保存章节、片段、概念和向量
标记任务成功
定时调度器
任务服务
arXiv 适配器
论文服务
PostgreSQL
后台任务执行器
解析/模型服务

5.5 智能问答过程


用户
React 前端
问答服务
检索服务
PostgreSQL/pgvector
LLM 服务
输入自然语言问题
POST /api/qa/questions
参数与权限校验
检索相关论文证据
关键词与向量检索
返回候选片段
返回排序后的证据
alt
[有可靠证据]
问题 + 论文片段 + 回答约束
生成回答
出处绑定与可信度检查
保存问答记录
答案、来源和内容定位
[无可靠证据]
未找到足够论文依据
展示结果
用户
React 前端
问答服务
检索服务
PostgreSQL/pgvector
LLM 服务

5.6 并发与任务控制

- Web 请求使用 FastAPI 的异步 I/O 能力处理网络和数据库操作；
- CPU 密集型 PDF 解析不得直接在请求处理协程中执行；
- 后台任务通过数据库任务表进行持久化，任务状态包括 `PENDING`、`RUNNING`、`SUCCESS`、`FAILED` 和 `CANCELLED`；
- 任务执行器领取任务时写入执行者和锁定时间，避免多个执行器重复执行；
- 抓取和解析操作必须具备幂等性；
- 对同一论文同一版本只允许存在一个有效解析任务；
- 外部请求统一设置连接超时、读取超时和最大重试次数。

---

6. 实现视图

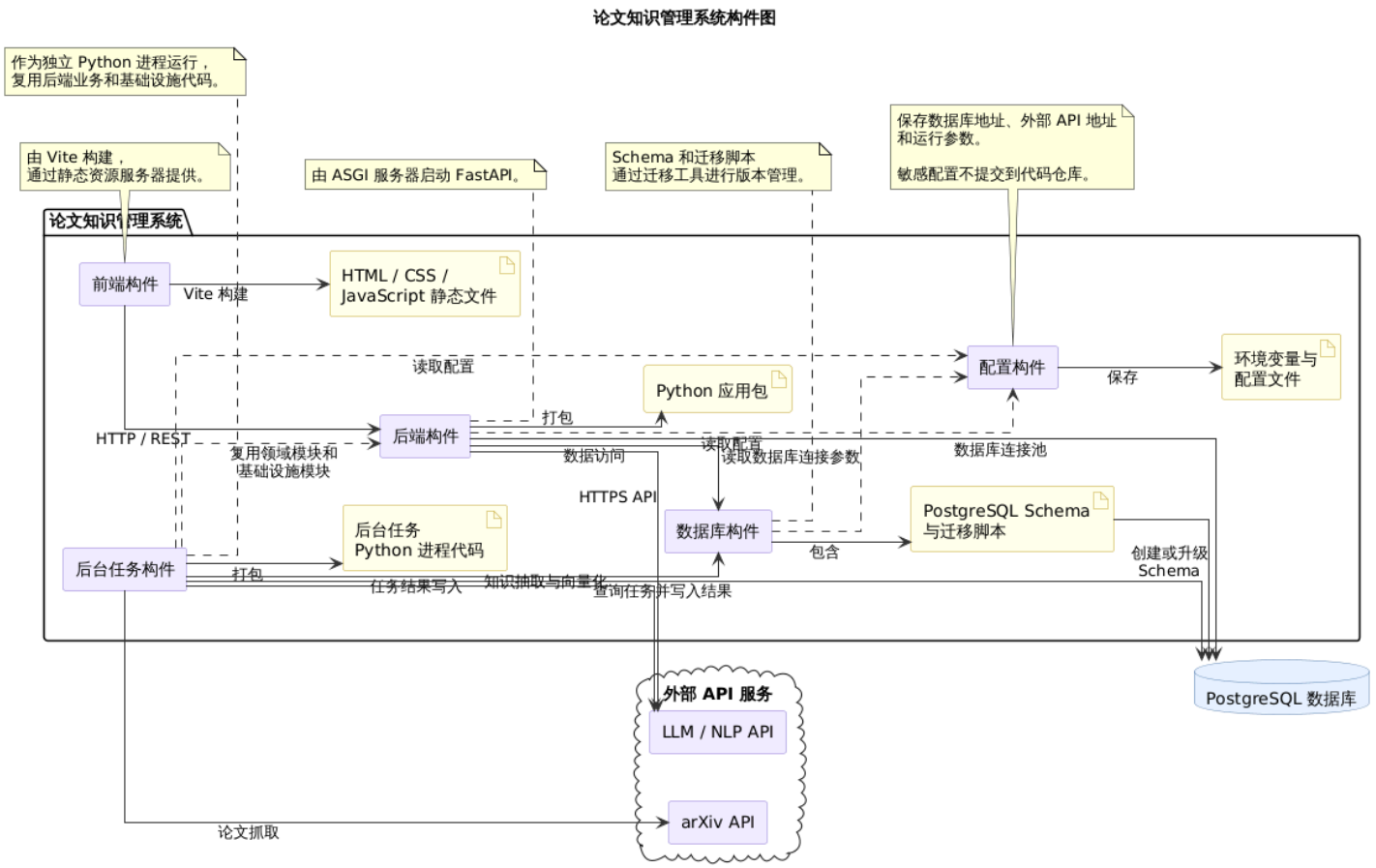
6.1 代码仓库结构

代码块

```
papermate/
├── frontend/
│   ├── src/
│   │   ├── api/                  # 后端 API 封装
│   │   ├── components/           # 通用组件
│   │   ├── layouts/              # 页面布局
│   │   ├── pages/                # 论文、问答、学习管理、管理页面
│   │   ├── routes/               # 路由配置
│   │   ├── stores/               # 全局状态
│   │   ├── types/                # 前端数据类型
│   │   └── utils/                # 工具函数
│   ├── public/
│   ├── package.json
│   └── vite.config.js
│
├── backend/
│   ├── app/
│   │   ├── api/
│   │   │   ├── routers/          # FastAPI 路由
│   │   │   ├── dependencies/     # 认证、数据库等依赖
│   │   │   └── schemas/          # 请求与响应模型
```

```
22  │   │   ├── application/
23  │   │   │   ├── paper/          # 论文用例编排
24  │   │   │   ├── search/         # 检索用例编排
25  │   │   │   ├── qa/             # 问答用例编排
26  │   │   │   ├── study/          # 学习管理用例编排
27  │   │   │   └── task/           # 任务用例编排
28  │   │   ├── domain/
29  │   │   │   ├── paper/
30  │   │   │   ├── knowledge/
31  │   │   │   ├── user/
32  │   │   │   ├── study/
33  │   │   │   └── task/
34  │   │   ├── infrastructure/
35  │   │   │   ├── database/       # ORM、仓储实现、数据库会话
36  │   │   │   ├── arxiv/          # arXiv 适配器
37  │   │   │   ├── parser/         # PDF/HTML 解析器
38  │   │   │   ├── llm/            # LLM 与嵌入模型适配器
39  │   │   │   ├── scheduler/      # 定时任务
40  │   │   │   └── logging/        # 日志配置
41  │   │   ├── workers/            # 后台任务执行器
42  │   │   ├── core/               # 配置、异常、安全和公共能力
43  │   │   └── main.py
44  │   ├── migrations/             # 数据库迁移
45  │   ├── tests/
46  │   │   ├── unit/
47  │   │   ├── integration/
48  │   │   └── api/
49  │   └── pyproject.toml
50  │
51  ├── docs/
52  │   ├── architecture/
53  │   ├── api/
54  │   └── diagrams/
55  ├── scripts/                    # 初始化、导入、备份等脚本
56  ├── .env.example
57  ├── docker-compose.yml          # 可选的本地开发环境
58  └── README.md
```

6.2 构件



构件	形式	说明
前端构件	静态 HTML/CSS/JavaScript 文件	由 Vite 构建，可由静态服务器提供。
后端构件	Python 应用包	由 ASGI 服务器启动 FastAPI。
后台任务构件	Python 进程	复用后端领域和基础设施模块。
数据库构件	PostgreSQL schema 与迁移脚本	通过迁移工具进行版本管理。
配置构件	环境变量和配置文件	保存数据库地址、外部 API 地址和运行参数，不保存到代码仓库。

6.3 团队并行开发边界

建议按照以下边界分工：

- 前端模块：页面、组件、路由和 API 接入；
- 论文抓取与任务模块：arXiv、调度、下载和任务状态；
- 解析与知识抽取模块：PDF、章节、概念、方法、结论和向量生成；
- 检索与问答模块：关键词检索、向量检索、问答编排和出处绑定；
- 数据库与学习管理模块：数据模型、迁移、收藏、笔记、阅读记录、权限和测试。

模块之间通过 API 契约、应用服务接口和数据库迁移协作，禁止多人同时随意修改公共模型。

---

7. 技术视图

7.1 技术选型

类别	技术	用途与理由
前端语言	JavaScript 或 TypeScript	实现浏览器端交互。若项目已经使用 JavaScript，可继续保持统一。
前端框架	React	适合组件化页面开发。
构建工具	Vite	提供快速开发服务器和前端构建。
UI 组件库	Ant Design	提供表格、表单、分页、布局和反馈组件。
后端语言	Python 3.x	具备丰富的 NLP、PDF 解析和模型调用生态。
Web 框架	FastAPI	提供类型校验、异步接口和自动 API 文档。
ASGI 服务器	Uvicorn	运行 FastAPI 应用。
ORM 与迁移	SQLAlchemy、Alembic	管理数据访问和数据库结构演化。
关系数据库	PostgreSQL	保存论文、用户、任务、学习记录和关系数据。

向量扩展	pgvector	在 PostgreSQL 中保存文本向量并执行相似度检索。
调度方式	APScheduler 或应用内调度器	定期创建抓取任务，适合 MVP 单实例部署。
PDF 解析	PyMuPDF、pypdf 或同类解析器	提取论文文本、页码和段落。
模型接口	可替换的 LLM/Embedding Provider	隔离具体云模型或本地模型实现。
测试工具	pytest、FastAPI TestClient、前端测试工具	支持单元、集成和 API 测试。
版本管理	Git、GitHub	分支协作、代码审查和版本追踪。

具体依赖版本由项目锁文件统一管理。所有成员必须使用一致的主要运行时版本，并通过 `package-lock.json`、`pyproject.toml` 或依赖锁文件固定依赖。

7.2 API 设计规范

- API 统一使用 `/api/v1` 前缀；
- 资源名称使用复数名词，例如 `/papers`、`/notes`、`/tasks`；
- 使用 HTTP 状态码表达请求结果；
- 列表接口统一支持分页、排序和筛选；
- 响应结构统一包含 `data`、`message` 和必要的分页信息；
- 错误响应统一包含错误码、提示信息和可选的字段错误；
- 对外部模型返回内容进行后端清洗，前端不直接调用模型服务；
- OpenAPI 文档作为前后端接口协作依据。

7.3 配置管理

以下配置不得写死在源代码中：

- 数据库地址、端口、用户名和密码；
- LLM 与嵌入服务地址、模型名称和 API Key；

- arXiv 请求间隔、分页大小和关键词；
- 文件存储目录；
- JWT 密钥和令牌有效期；
- 日志级别、任务并发数和超时参数。

开发环境通过 `.env` 加载配置，仓库中只保留 `.env.example`。

---

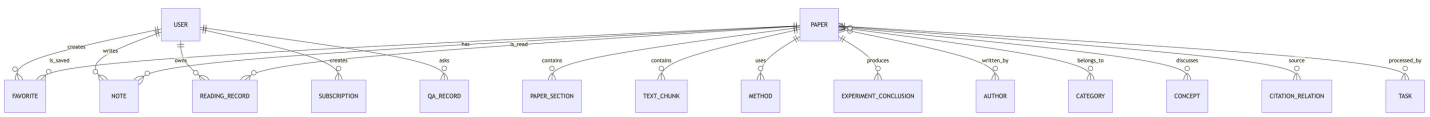
8. 数据视图

8.1 核心数据实体

实体	主要字段	说明
User	id、username、email、password_hash、status	系统用户。
Role	id、name	用户角色。
Paper	id、arxiv_id、version、title、abstract、published_at、pdf_url、status	论文主记录。
Author	id、name、organization	作者信息。
Category	id、code、name	arXiv 类别或系统主题。
PaperDocument	id、paper_id、file_path、checksum、parse_status	论文文件及解析状态。
PaperSection	id、paper_id、title、level、page_start、page_end	论文章节。
TextChunk	id、paper_id、section_id、content、position、embedding	检索和问答使用的文本片段。

Concept	id、name、description	核心概念。
Method	id、paper_id、name、description	论文使用的方法。
ExperimentConclusion	id、paper_id、content	实验结果或结论。
CitationRelation	source_paper_id、target_paper_id、relation_type	论文引用或关联。
Favorite	user_id、paper_id、created_at	收藏关系。
Note	id、user_id、paper_id、content、visibility	用户笔记。
ReadingRecord	id、user_id、paper_id、progress、last_read_at	阅读历史和进度。
Subscription	id、user_id、query、frequency、enabled	主题订阅。
QARecord	id、user_id、question、answer、sources、created_at	问答历史。
Task	id、type、status、payload、retry_count、error_message	后台任务。

8.2 核心关系



8.3 数据存储策略

1. 论文元数据、用户数据、学习记录和任务状态使用 PostgreSQL 关系表；
2. 论文文本片段向量使用 pgvector 字段；
3. PDF 原始文件和生成的 Markdown 文件在 MVP 阶段存放于后端本地文件目录，数据库保存路径、校验值和状态；
4. JSON 字段只用于结构变化较大的辅助数据，不代替核心关系表；

5. 重要关系使用外键约束；
6. 删除用户或论文时优先使用逻辑删除，避免破坏引用关系和审计记录；
7. 密码只保存安全哈希，不保存明文；
8. 笔记、问答历史等用户数据必须通过用户 ID 进行访问控制。

8.4 索引设计

- `paper.arxiv_id + version`：唯一索引，用于去重；
- `paper.title`：全文或模糊检索索引；
- `paper.published_at`、`paper.status`：普通索引；
- 作者、类别和论文关联表：联合索引；
- `favorite.user_id + paper_id`：唯一联合索引；
- `task.status + task.type + created_at`：任务领取索引；
- `text_chunk.embedding`：向量索引；
- `text_chunk.paper_id + section_id + position`：内容定位索引。

8.5 数据一致性

- 论文和作者、类别关系在同一事务中写入；
- 抓取任务和解析任务使用唯一业务键防止重复创建；
- 解析新版本时先写入新结果，完成后再切换论文当前版本；
- 任务失败不删除已有有效论文数据；
- 收藏、笔记和阅读记录必须校验用户对目标数据的访问权限。

---

9. 算法视图

9.1 论文去重算法

优先级从高到低：

1. 使用标准化后的 `arxiv_id + version` 进行精确匹配；

2. 若缺少标准 ID，则使用 PDF 校验值匹配；
3. 使用标准化标题、第一作者和发表年份进行辅助匹配；
4. 对标题执行大小写、空白、标点和特殊字符归一化；
5. 对疑似重复但无法确定的论文标记为待人工确认，不直接删除。

9.2 文本解析与分块算法

1. 下载并校验 PDF 文件；
2. 提取页级文本和位置信息；
3. 识别标题、摘要、章节和参考文献区域；
4. 按章节、段落和最大长度切分文本；
5. 为相邻片段保留少量重叠，减少语义断裂；
6. 保存论文 ID、章节、页码、片段序号和字符范围；
7. 对片段批量生成嵌入向量；
8. 将向量及定位信息写入 pgvector。

9.3 知识抽取算法

系统从论文中抽取以下结构化内容：

- 研究问题；
- 核心概念；
- 研究方法；
- 数据集和实验设置；
- 实验结果；
- 主要结论；
- 论文贡献；
- 已知限制。

抽取流程：

1. 根据章节类型选择候选文本；
2. 使用规则、NLP 模型或 LLM 生成结构化结果；
3. 对返回结构进行字段校验；

4. 合并重复概念和方法；
5. 保存内容来源位置；
6. 对低置信度结果标记为待复核。

9.4 混合检索算法

1. 对用户查询进行文本清洗；
2. 执行标题、摘要、作者、类别和标签关键词检索；
3. 生成查询向量并执行向量相似度检索；
4. 对两类结果进行分数归一化；
5. 根据关键词分数、向量相似度、时间和论文状态计算综合分数；
6. 去除同一论文中的过多重复片段；
7. 返回前 K 个论文或文本片段。

综合得分可表示为：

代码块

```
score = α × keyword_score
      + β × vector_score
      + γ × freshness_score
      + δ × metadata_match_score
```

其中参数由测试结果调整

---

10. 性能视图

系统用户量和并发量较小，暂无明确的响应时间、吞吐量等指标

---

11. 可靠性视图

系统不是关键业务系统，暂无高可用、容灾和故障恢复要求

---

12. 安全性视图

仅涉及基本登录和权限管理，不存在复杂或高等级安全需求

---

13. 易用性视图

界面和操作流程较简单，相关内容可在界面设计中说明

---

14. 可维护性视图

已通过分层架构、模块划分和接口设计体现，无需单独成图